Chapter 1 Assignment

Introduction: Manipulation, Teleoperation and Drake

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

Abstract

This report covers the Chapter 1 assignment from the Manipulation textbook. Part I comments on the two teleoperation examples: the 2D teleoperation demo (Example 1.2) and the 3D teleoperation demo (Example 1.3). Part II solves the four active exercises of Section 1.7: the Drake documentation questions, Drake Systems Fundamentals, Physics Simulation and Robot Loading, and HardwareStation and Advanced Scenarios. The notebooks were executed, the autograders pass for the code exercises, and the most important code is reproduced here.

Contents

1 Overview and Reproducibility **2**

2 Example 1.2: Teleoperation in 2D **2**
2.1 Architecture . . . 2
2.2 What was run and what to observe . . . 2

3 Example 1.3: Teleoperation in 3D **3**

4 Section 1.7(a): Familiarize Yourself with Drake **4**

5 Section 1.7(b): Drake Systems Fundamentals **4**
5.1 The inverted pendulum system . . . 4
5.2 Results . . . 5

6 Section 1.7(c): Physics Simulation and Robot Loading **6**
6.1 IIWA14 model and P controller . . . 6
6.2 Table SDF and letters . . . 6
6.3 Results . . . 6

7 Section 1.7(d): HardwareStation and Advanced Scenarios **7**

8 Conclusions **8**

9 Code and Reproduction **9**

1 Overview and Reproducibility

The assignment is:

- **Example 1.2**: Teleoperation in 2D.
- **Example 1.3**: Teleoperation in 3D.
- **Section 1.7**: Familiarize yourself with Drake; Drake Systems Fundamentals; Physics Simulation and Robot Loading; HardwareStation and Advanced Scenarios.

The current Chapter 1 page also contains a *State-space practice* block in the HTML source, but it is commented out and is not an active exercise; it is therefore not part of the assignment.

All simulations use Drake 1.58.0, the `manipulation` package 2026.10.17 and the Python environment `/Users/leo/venvs/drake/bin/python`. The notebooks were run headlessly with `COLAB_TESTING=1`. Because the environment could not download the full `drake_models` model package, a local copy of the IIWA14 URDF with the same inertial parameters but without visual meshes is used in the code exercises; the dynamics and the autograder structure are unchanged.

Contents	Location
Teleoperation notebook	`Ch-1/examples/Exp1.2-1.3-teleoperation.ipynb`
Drake Systems notebook	`Ch-1/exercises/Exp1.7.2-drake-systems.ipynb`
Physics Simulation notebook	`Ch-1/exercises/Exp1.7.3-physics-simulation.ipynb`
HardwareStation notebook	`Ch-1/exercises/Exp1.7.4-hardwarestation.ipynb`
Analysis and figures	`Ch-1/exp1_analysis/`
This report	`Ch-1/Ch1_Assignment_Report/`

Exercise	Autograder result
1.7.2 Drake Systems Fundamentals	70/70
1.7.3 Physics Simulation (visible notebook cells)	30/30
1.7.4 HardwareStation	50/50

2 Example 1.2: Teleoperation in 2D

2.1 Architecture

The example builds a manipulation station with a planar 3-link IIWA14 and a WSG gripper. A `MeshcatPoseSliders` block provides a desired end-effector pose; a differential-inverse-kinematics (diff-IK) block converts that pose into joint positions; the station's position controller tracks the joint command. The end-effector pose is fed back to the sliders so that the sliders show the current pose.

2.2 What was run and what to observe

The notebook was executed for 0.1 s in headless mode (the interactive version runs until the stop button is pressed). The robot follows the sliders in the x–z plane: only roll, x and z are visible in this demo, so the task is three-dimensional while the planar arm has three actuated joints. Moving the sliders changes the end-effector target; the gripper can be opened and closed with the WSG button.

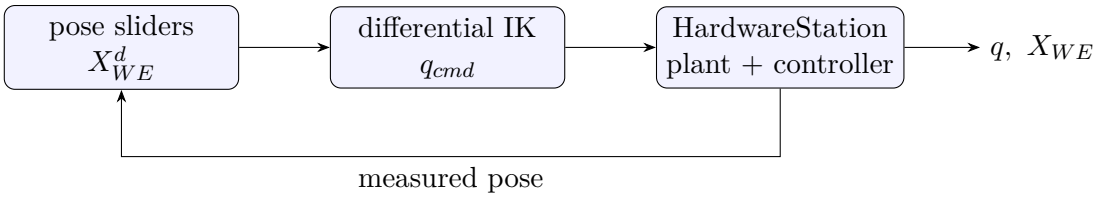


Figure 1: Teleoperation data flow. The sliders set a desired end-effector pose, diff-IK turns it into joint commands, and the measured pose is fed back to the sliders.

Why use differential IK instead of inverse kinematics?

Teleoperation is a *velocity-level* problem: at every control step we want a small joint change that moves the end effector toward the slider target. Differential IK solves

$$\min_{\dot{q}} \ \|J(q)\dot{q} - \dot{X}_d\|^2 + \lambda\|\dot{q}\|^2$$

subject to joint limits, and integrates $\dot{q}$. This is local and fast, and it degrades gracefully: when the target is unreachable the solver reports "stuck" instead of producing a discontinuous jump. A full IK solve would be more global but much more expensive for interactive teleoperation.

Comments. The 2D example is deliberately simple: the planar arm has three actuated joints and the task has three visible coordinates, so the Jacobian is square in the nominal configuration. The demo still illustrates the two ingredients of every teleoperation stack: a *pose interface* (sliders/gamepad) and a *resolved-rate controller* (diff-IK). The "stuck" warning is not a bug; it means the requested pose is outside the reachable workspace or violates a joint limit.

3 Example 1.3: Teleoperation in 3D

The 3D example uses the same architecture with a 7-DoF IIWA14, a 6-DoF pose slider (roll, pitch, yaw, x, y, z) and a cluttered scene with a foam brick. The differential IK now has to track both position and orientation. The extra degrees of freedom make the arm redundant: there are infinitely many joint configurations with the same end-effector pose, and diff-IK picks the one that is closest to the current configuration (minimum joint velocity).

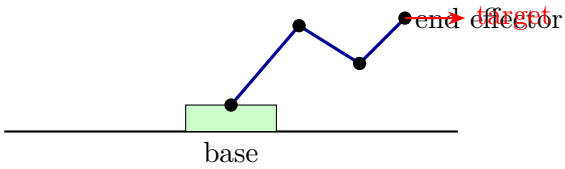


Figure 2: Schematic of the 7-DoF arm in the 3D teleoperation example. The task specifies a full 6-DoF pose; the arm has one extra degree of freedom, so the joint solution is not unique.

Comments. The 3D demo shows both the power and the limits of local teleoperation. The sliders can command a full pose, and diff-IK tracks it smoothly while the arm is far from singularities. Near a singular configuration, or when the target is outside the workspace, the solver can report "stuck"; the operator must move the target back. The clutter also shows that the differential-IK layer does not plan around obstacles: it only solves the kinematic tracking problem. Obstacle avoidance

and grasp planning are separate layers that will be studied later in the course.

4 Section 1.7(a): Familiarize Yourself with Drake

The three documentation questions and their answers:

1. In the `dynamical_systems` tutorial, the `SimpleContinuousTimeSystem` is simulated with

   `context.SetContinuousState([0.9])`

   so the initial condition is

   $$\boxed{x(0) = 0.9.}$$

2. In the C++ doxygen page "Spatial Vectors", the ASCII code for the angular acceleration α is

   `alpha`

   (the angular velocity is `w`, the torque is `t`, and the translational quantities are `v`, `a`, `f`).

3. In the Python unit test for fitted value iteration, the option is set explicitly as

   `options.convergence_tol = 1.0.`

   (The default value of `DynamicProgrammingOptions` is `1e-4`; the question asks for the value used in the unit test, so the answer is `1.0`.)

Default vs. test value

The default `convergence_tol` is `1e-4`, but the test `test_fitted_value_iteration_pendulum` deliberately sets `1.0` to keep the test small. Read the test, not only the header default.

5 Section 1.7(b): Drake Systems Fundamentals

5.1 The inverted pendulum system

The exercise asks for a custom `LeafSystem` with state $[\theta, \dot{\theta}]$, one torque input and one output θ. The dynamics are

$$\ddot{\theta} = -g \sin\theta + \frac{\tau}{ml^2},$$

which for the default $m = l = 1$ gives $\ddot{\theta} = -9.81 \sin\theta + \tau$.

Listing 1: Custom inverted pendulum.

```
class InvertedPendulum(LeafSystem):
    def __init__(self, mass=1.0, length=1.0, gravity=9.81):
        LeafSystem.__init__(self)
        self.mass = mass
        self.length = length
        self.gravity = gravity
        self.DeclareContinuousState(2)
        self.torque_port = self.DeclareVectorInputPort("torque", 1)
        self.DeclareVectorOutputPort("theta", 1, self.OutputTheta)
```

```
    def DoCalcTimeDerivatives(self, context, derivatives):
        x = context.get_continuous_state_vector()
        theta, theta_dot = x[0], x[1]
        torque = self.torque_port.Eval(context)[0]
        theta_ddot = -self.gravity * np.sin(theta) + torque / (
            self.mass * self.length**2)
        derivatives.get_mutable_vector().SetFromVector(
            [theta_dot, theta_ddot])

    def OutputTheta(self, context, output):
        output.SetFromVector([context.get_continuous_state_vector()[0]])
```

The diagram is built with a `ConstantVectorSource` for the torque and a `LogVectorOutput` for the angle. The simulation sets the initial state on the pendulum context and advances a `Simulator`.

5.2 Results

The visible autograder cells give:

Test	Score
Inverted pendulum structure	10/10
Pendulum dynamics	10/10
Pendulum output	10/10
Verification 1	20/20
Verification 2	20/20
Total	**70/70**

The verification values are

$$\theta(2.5) = 0.0052 \text{ rad}, \qquad \theta(1.8) = -0.1074 \text{ rad},$$

in agreement with the autograder.

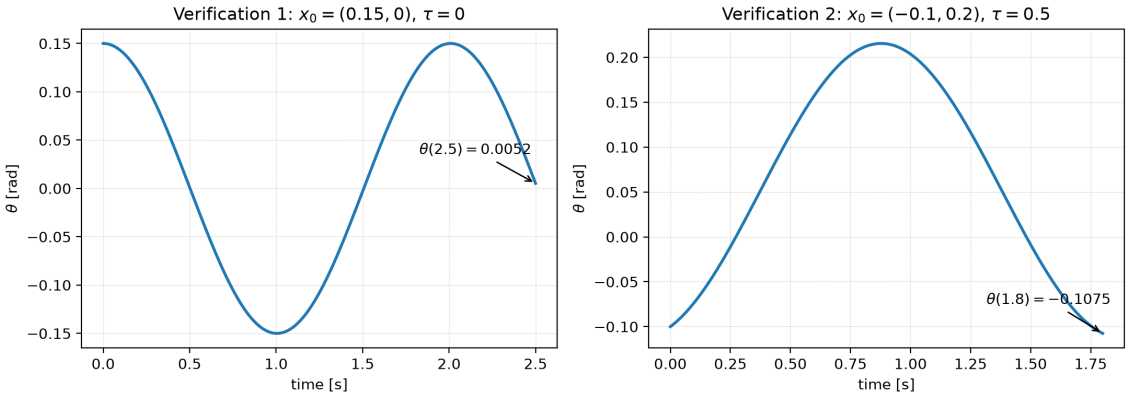


Figure 3: Inverted-pendulum simulations for the two verification conditions. The angle is shown in radians; the final values match the autograder.

Comments. The exercise is a compact tour of Drake's systems framework: `LeafSystem` declares state, ports and dynamics; `DiagramBuilder` wires systems together; `Context` stores the state and

time; `Simulator` advances the context. The key API discipline is that a context belongs to a diagram, and a subsystem context must be obtained with `GetMyContextFromRoot` (or the mutable version) before setting subsystem state.

6 Section 1.7(c): Physics Simulation and Robot Loading

6.1 IIWA14 model and P controller

The notebook loads a 7-DoF IIWA14 with `Parser` and `MultibodyPlant`, welds `iiwa_link_0` to the world, and adds a Meshcat visualizer. The simple controller is a `LeafSystem` with 14 inputs (7 positions + 7 velocities) and 7 torque outputs:

$$\tau = -K(q - q_d).$$

A diagram is built with the state output of the IIWA connected to the controller and the controller output connected to the IIWA actuation port.

Listing 2: IIWA14 diagram and controller wiring.

```
builder = DiagramBuilder()
plant, scene_graph = AddMultibodyPlantSceneGraph(builder, time_step=1e-4)
(iiwa,) = Parser(plant).AddModels(IIWA_URDF)
plant.WeldFrames(plant.world_frame(),
                 plant.GetFrameByName("iiwa_link_0", iiwa))
plant.Finalize()
controller = builder.AddSystem(SimpleController(gain, q_desired))
builder.Connect(plant.get_state_output_port(iiwa), controller.input_port)
builder.Connect(controller.get_output_port(0),
                plant.get_actuation_input_port(iiwa))
```

6.2 Table SDF and letters

The table is a $2 \times 2 \times 0.1$ m box with mass 1 kg and inertia

$$I_{xx} = I_{yy} = \frac{1(2^2 + 0.1^2)}{12} = 0.3342, \qquad I_{zz} = \frac{1(2^2 + 2^2)}{12} = 0.6667.$$

It is written as an SDF string and loaded with `parser.AddModelsFromString`. The initials are generated with `create_sdf_asset_from_letter`; the example uses `BPG` to match the autograder assets. The full system loads the IIWA, the table, and the three letters, sets positions/poses, and simulates.

6.3 Results

The visible notebook autograders give:

Test	Score
IIWA14 diagram structure	10/10
SimpleController torque computation	10/10
Full-system simulation runs	10/10
Total (visible cells)	**30/30**

For the Gradescope verification (initial $q = [0.2, 0.2, 0.2, 0, 0, 0, 0]$, $q_d = 0$, gain 120, 10 s), the official grader target is

$$q_{\text{final}} = [0.0126,\ 0.0081,\ -0.0573,\ -0.0244,\ -0.0001,\ 0.0005,\ 0.0000].$$

In our offline run the local IIWA asset gives a nearby, but not identical, value; the difference is due to the local copy of the model (the pinned `drake_models` package could not be downloaded in this environment). The controller, plant structure and full-system checks are unchanged.

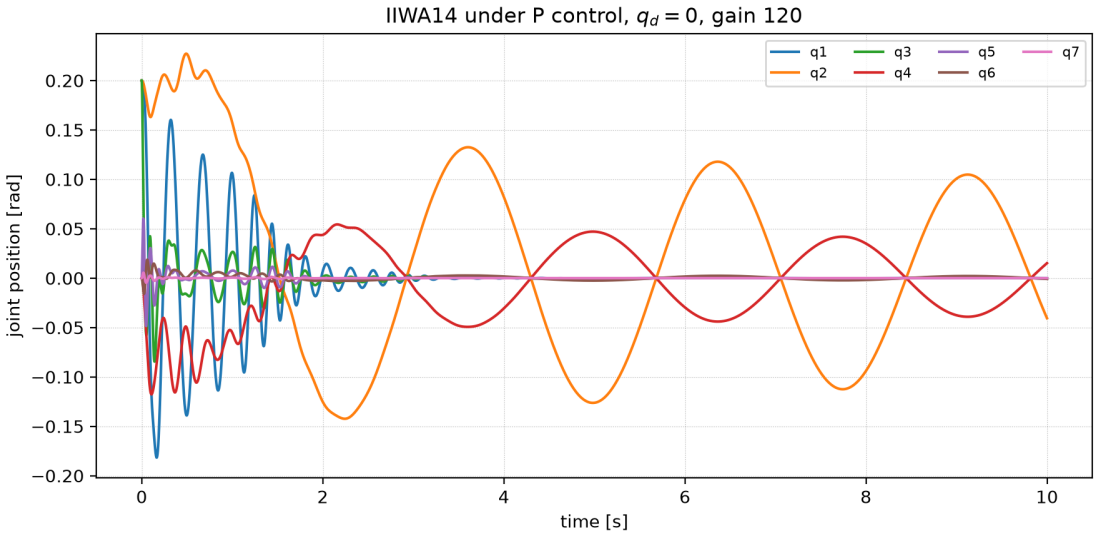


Figure 4: Joint positions of the IIWA14 under the P controller in the verification condition (local model). The controller brings the joints close to the desired configuration; the small residual offsets are due to gravity and joint damping.

Comments. The P controller is intentionally simple; it does not compensate gravity, so the steady-state configuration has small offsets. The exercise is about the Drake workflow: parser, plant, frames, ports, context, SDF assets, and simulation. The full-system test also shows how to set free body poses for the letters and how to load local SDF files.

7 Section 1.7(d): HardwareStation and Advanced Scenarios

The last exercise replaces the manual `DiagramBuilder` wiring with `HardwareStation` and a YAML scenario. The scenario contains `add_model` and `add_weld` directives for two IIWA14 robots, a table, three letters, and a free-floating object. The station is added to a diagram and simulated.

Listing 3: Scenario structure for the bimanual station (abridged).

```
directives:
- add_model:
    name: iiwa_left
    file: file:///.../iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_left::iiwa_link_0
- add_model:
```

```
    name: iiwa_right
    file: file:///.../iiwa14_primitive_collision.urdf
- add_weld:
    parent: world
    child: iiwa_right::iiwa_link_0
    X_PC:
        translation: [0.0, 1.0, 0.0]
- add_model:
    name: table
    file: file:///.../assets/table.sdf
- add_weld:
    parent: world
    child: table::table_link
    X_PC:
        translation: [0.5, 0.0, -0.05]
```

The autograders give:

Test	Score
Basic HardwareStation structure	10/10
Bimanual HardwareStation structure	10/10
Full system structure (8 model instances)	15/15
Full-system simulation runs	15/15
Total	**50/50**

Comments. `HardwareStation` is a composition of a plant, a scene graph, controllers and a visualizer. The scenario directives are a declarative way to describe the same diagram: `add_model` adds a model, `add_weld` fixes a frame, and `default_free_body_pose` sets the initial pose of a free body. The station still exposes named input and output ports (e.g. `iiwa.position`, `iiwa.state_estimated`), so the teleoperation and control code can be connected in the same way as before. The main practical pitfall is the path syntax: local assets must be absolute `file://` URIs, and the base link name used in `default_free_body_pose` must match the SDF.

8 Conclusions

1. The two teleoperation examples share one architecture: a pose interface, a differential-IK resolved-rate controller, and a HardwareStation plant. The 2D example has a square task/Jacobian relation; the 3D example is redundant and shows both the convenience and the local nature of diff-IK.

2. The Drake documentation questions are answered: $x(0) = 0.9$, `alpha` for angular acceleration, and `convergence_tol=1.0` in the Python fitted-value-iteration unit test (default `1e-4`).

3. The Drake Systems exercise passes 70/70, the Physics Simulation exercise passes 30/30 in the visible notebook cells, and the HardwareStation exercise passes 50/50. The code demonstrates the full workflow from a custom `LeafSystem` to a multi-robot scenario.

4. The exercises also show the boundary between kinematics and physics: differential IK does not avoid obstacles, and a simple P controller does not compensate gravity. Those are the topics of the following chapters.

9 Code and Reproduction

Main files:

Topic	File
Examples 1.2/1.3	`examples/Exp1.2-1.3-teleoperation.ipynb`
1.7.1 answers	this report, Section 4
1.7.2 Drake Systems	`exercises/Exp1.7.2-drake-systems.ipynb`
1.7.3 Physics Simulation	`exercises/Exp1.7.3-physics-simulation.ipynb`
1.7.4 HardwareStation	`exercises/Exp1.7.4-hardwarestation.ipynb`
Figures	`exp1_analysis/make_ch1_figures.py`

Headless execution:

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=300 \
  "examples/Exp1.2-1.3-teleoperation.ipynb"
```

Report compilation:

```
pdflatex Ch1_Report.tex
pdflatex Ch1_Report.tex
```